mode which means no other transactions can access the resource while the lock is in effect. How can you make any change to a table then for a busy database?

Don't worry. Most of the time the lock durations are short and clients can wait a bit if needed.

Some changes are less safe though, particularly for tables with very high row counts and queried heavily. In this chapter you'll learn to identify which changes are the most risky, and how to work around those risks using safer tactics.

Besides changes to the database structure, you may be conducting large scale *backfill* operations. Backfills populate one or more columns that didn't exist before and are otherwise null, to have useful values for the application.

Backfilling on tables with millions or billions of rows without stopping the server is another type of technical challenge that will be covered in this chapter.

Take a look at some of the terminology you'll be learning.

Busy Databases Terminology



- Strong Migrations — Library that adds more safety to Migrations
- Multiversion Concurrency Control (MVCC) — PostgreSQL mechanism for managing row modifications and concurrent data access
- ACID — Design properties of Atomicity, Consistency, Isolation, Durability
- Isolation Levels — Configurable access level among transactions accessing the same data
- Denormalization — Duplicating some column data
- Backfilling — Populating a recently added empty column on existing table rows
- Table Rewrites — Internal changes from some schema modifications that can cause a significant availability delay

Earlier you learned that some types of Migrations are riskier than others. How can you find the riskier ones?